Distributed Edge AI Systems

Fei Hu, Kunal Mehta, Shivakant Mishra
University of Colorado
Boulder, USA
Fei.Hu|Kunal.Mehta|mishras@colorado.edu

Mohammad AlMutawa
Kuwait University
Khaldiya, Kuwait
malmutawa@cs.ku.edu.kw

Abstract

Edge computing has become a promising computing paradigm for building IoT (Internet of Things) applications, particularly applications with latency and privacy constraints. However, these applications typically tend to be compute-intensive and compute resources are limited at the edge when compared to the cloud, so it is important to efficiently utilize all computing resources available at the edge. A key challenge in utilizing these resources is the scheduling of different computing tasks in a dynamically varying, highly hybrid computing environment. We describe the design, implementation, and evaluation of a dynamic distributed scheduler for the edge that constantly monitors the current state of the computing infrastructure and dynamically schedules various computing tasks to ensure that all application constraints are met in another paper. Based on that, this paper mainly proposes a profile evaluation method and results when applying an augmented reality application on distributed systems at the edge. With that work done, we propose and implement a good solution to efficiently distribute edge AI applications at the edge.

Keywords: Edge Computing, AI Applications, Distributed Systems, Dynamic Distributed Scheduler, Containers

ACM Reference Format:
Fei Hu, Kunal Mehta, Shivakant Mishra and Mohammad AlMutawa. 2023. Distributed Edge AI Systems. In *2023 IEEE/ACM 16th International Conference on Utility and Cloud Computing (UCC '23), December 4–7, 2023, Taormina (Messina), Italy*. ACM, Taormina, Italy, 6 pages. https://doi.org/10.1145/3603166.3632546

1 Introduction

In recent years, Edge AI systems [15, 16] have emerged as a promising solution for handling the growing demand for AI workloads by bringing computation closer to the data sources and reducing latency [5, 19]. However, constructing AI applications at the edge presents a significant challenge due to their high computational demands and the limited computing resources available at edge servers compared to cloud infrastructure. To tackle this challenge, developers have been adopting an ad hoc approach, distributing computational tasks across diverse nodes at the edge, including low-end sensing devices and processing units. However, this task distribution strategy necessitates data transfer between nodes, incurring additional costs. Achieving optimal performance requires striking a delicate balance between the advantages of parallel computation across nodes and the drawbacks of increased communication delays. This balance is further complicated by dynamic changes in network conditions, resulting in varying communication delays, as well as fluctuations in CPU load on computing nodes, leading to dynamic changes in task execution times. Additionally, the highly diverse nature of edge compute nodes, with variations in compute, storage, and communication capabilities, adds complexity to this optimization process.

We propose a new distributed scheduling method that addresses this challenge by leveraging the computing capacities of all nodes, dynamically adjusting the scheduling decisions based on the system's current state, and taking various constraints into account[12]. These constraints include real-time requirements, computation capability, network bandwidth, and other factors that impact the performance of the system.

In our paper [12], although face detection applications are not extensively utilized, we introduce an augmented reality application where object detection operates on video streams. Instead of consolidating all services onto a single computing node, we dissect this application into three primary components: frame extraction, object detection, and frame combination. These components are distributed across computing nodes to enhance performance. The frame extraction module transforms the video into frames, which are then concurrently dispatched to multiple object detection servers via threading. These servers host object detection algorithms tasked with identifying objects within the frames provided by the frame extraction service. Lastly, the frame combination container merges the detected objects, producing the final output. Each of these services is encapsulated within a Docker container, and all services seamlessly integrate within our Kubernetes cluster.

This paper makes the following significant contributions:

- Deployment of Augmented Reality in a Distributed System: The paper outlines a method for deploying an augmented reality application within a distributed system.



This work is licensed under a Creative Commons Attribution International 4.0 License.
UCC '23, December 4–7, 2023, Taormina (Messina), Italy
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0234-1/23/12.
https://doi.org/10.1145/3603166.3632546

- Implementation Using Kubernetes: The authors have implemented this approach utilizing Kubernetes, a popular container orchestration system.
- Evaluation Across Diverse Scenarios: The paper conducts a comprehensive evaluation of this system across various scenarios. This evaluation serves as a foundation for potential integration with the dynamic distributed scheduler[12].

2 Related Works

End devices are capable of handling some tasks on their own with the available computation resources. The scheduling problem at the edge involves distributing tasks to these nodes. However, there are several challenges currently faced in scheduling, including incomplete information[22][20], constraint cooperativeness , task and trust constraints [9][3], dynamic environment [18][17].

To address these challenges, current research in edge computing focuses on meeting objectives such as low latency, low energy consumption, and multi-objectives. These efforts primarily fall into two categories. The first is a centralized solution where a scheduler manager offloads tasks to edge devices with limited memory, battery, and computational power[25][24]. The second category is a distributed solution where multiple nodes coordinate to schedule tasks. Numerous solutions and protocols have been designed in this direction. For instance, Ahmad et al. proposed a game theory-based solution in their paper [1] to optimize the solution and save energy. Bertuccelli et al. designed a decentralized task allocation protocol under dynamic environments in their paper [2]. Zheng et al. presented an integrated formulation for optimizing task placement in distributed systems in their paper [23]. This method considers task priority to meet end-to-end deadline constraints and minimize total latencies. Paper [6, 7, 14] discusses how end devices can extend their processing scope and complete user requests by collaborating with other access nodes.

However, all of this research is based on mathematical modeling, which is unable to fully account for the complexity and variability of dynamic edge networks. Therefore, we propose a brand-new solution: a dynamic distributed scheduling algorithm based on real-world evaluation, to support computing at the edge.

Current research focuses on the distributed deep learning model. Paper[21] discusses the challenges and opportunities of distributed machine learning over conventional (centralized) machine learning, including techniques used for distributed machine learning and an overview of available systems. Distributed machine learning is a technique used to improve the performance, accuracy, and scalability of machine learning algorithms for larger input data sizes[21]. There are three primary categories of systems for performing machine learning tasks in a distributed environment: database, general, and purpose-built systems. this paper [13] highlights the growing threat of malware in the internet era and proposes a novel framework for detecting malware using machine learning algorithms in a distributed environment. Paper[4] explores the potential benefits of distributing the training process on a distributed GPU cluster, analyzing both the scalability of the task and its performance in the distributed setting and the impact of distributed training methods on the final accuracy of the models. In addition to this, article [8] describes a software framework called DistBelief that can train deep neural networks with billions of parameters using thousands of machines.

Our focus would not be on integrating distributed computing within the deep learning model, but rather on utilizing a distributed architecture to effectively harness the available computing resources and produce optimal outcomes.

3 Dynamic Distributed Scheduler

The core principle guiding this scheduling algorithm is to minimize device communication by prioritizing the local node for handling tasks if it has the available capacity. Adhering to this principle, our system employs a distributed architecture comprising of *two levels*. Every node, including edge servers, end devices, or the cloud, hosts a scheduler component. The edge server's scheduler functions as the central coordinator. On the lower level, each device periodically assesses its current computing status, including CPU load, network latency to the edge server, and remaining battery power (if applicable), and communicates this data to the edge server. On the upper level, the edge server maintains a real-time status record for each device based on this information and utilizes it for decentralized scheduling decisions. Preemptive device profile evaluations have been conducted and shared with the edge server to facilitate scheduling decisions. Taking various objectives and constraints into account, this algorithm presents a promising strategy for optimizing the deployment of AI workloads in practical scenarios. The dynamic distributed scheduler is like Figure 1

The experimental results provide evidence for the effectiveness of the DDS algorithm in comparison to other scheduling algorithms. Comparing the performance of running all tasks on the edge server versus running them on a Raspberry Pi 1 highlights the importance of a distributed scheduling mechanism. Additionally, comparing the efficiency of static versus dynamic distributed scheduling indicates the superiority of the dynamic approach. Please refer to paper[12] for more information.

4 AI Applications Using Distributed Systems

To integrate with the dynamic distributed scheduler, we need to get the device profile for this augmented reality application. This device profile represents the computing

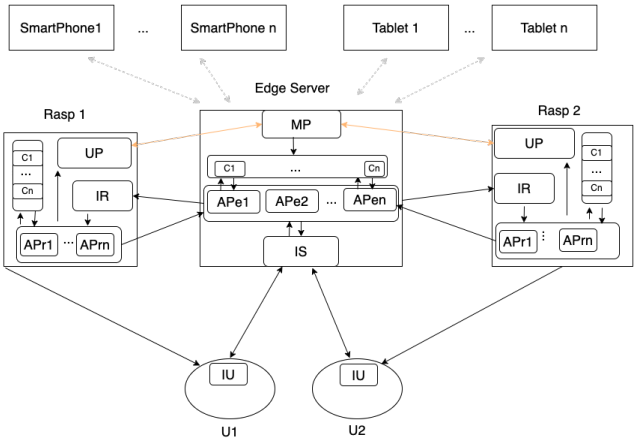


Figure 1. Dynamic Distributed Scheduler Architecture

capacity for a single application under varying dynamic CPU loads. This section will introduce how to evaluate it under different scenarios.

4.1 Methodology

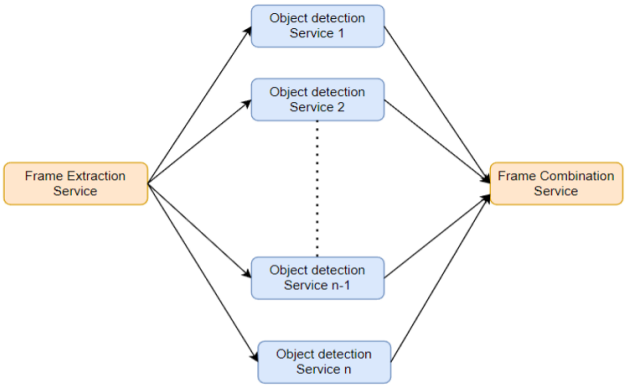


Figure 2. Distributed System Architecture

Our system operates with a division into three distinct services, each with its own set of responsibilities. The distributed system architecture is as shown in Figure 2

Firstly, we have the Frame Extraction Service, which is responsible for the conversion of video files into a series of images. These images are then sent to multiple object detection servers simultaneously via threading, depending on the number of servers available. Our mode of communication for sending the images to the object detection servers is REST where these images are pickled and sent as POST requests.

Next, we have the Object Detection Service, which houses the object detection models responsible for detecting objects in the series of images generated by the Frame Extraction Service. We make use of SSD $MobileNet_v3$ to carry out the task of object detection with precision and accuracy. The SSD MobileNetV3[10] are neural networks designed for use on mobile devices such as smartphones. The new MobileNetV3 models have been developed through a blend of automated search algorithms and innovative architectural designs, tailored to excel in both high-resource and low-resource scenarios. The results show that MobileNetV3-Large is 3.2%, more accurate on ImageNet classification while reducing latency by 20% compared to MobileNetV2[11], and MobileNetV3-Small is 6.6% more accurate compared to a MobileNetV2 model with comparable latency. MobileNetV3-Large detection is over 25% faster at roughly the same accuracy as MobileNetV2 on COCO detection. MobileNetV3-Large LRASPP is 34% faster than MobileNetV2 R-ASPP at similar accuracy for Cityscapes segmentation. Overall, the new MobileNet models achieve state-of-the-art results for mobile classification, detection, and segmentation. Given our need for high speed and the fact that we will be utilizing RaspberryPi, we have decided to employ the SSD MobileNetV3 model.

Finally, there is the Frame Combination Service, responsible for merging the images produced by various object detection services and presenting them to the user as a seamless live video stream. Pygame is utilized for implementation, where a fixed window size is maintained, ensuring images are correctly ordered and displayed to the user.

Each component within our system fulfills a vital role, ensuring the efficiency and precision of our object detection system. Through their collaborative efforts, these services facilitate seamless object detection, ensuring a superior user experience. Each service is encapsulated within a Docker container, and the entire system is interconnected through Kubernetes, enhancing its scalability and manageability.

4.2 Experiment Results

We have different environments where we performed our experiments and generated different results. In our experiments, we utilized various environments to produce distinct results. The specific configurations of the devices used are outlined in Table 1.

4.2.1 Case 1: All on the edge server. Figure 3 shows how all services are connected and deployed on an edge server. By increasing the number of object detection containers to simulate dynamic CPU load, we get the average time for object detection.

Figure 4 shows the correlation between the number of object detection containers and the average time taken to detect objects in frames with different video quality. The data showcases a strong relationship between these two variables, and as the number of object detection containers increases, the time taken to detect objects in frames decreases, indicating optimal performance of the system.

When the video quality is 480p, the average time required for a single object detection container to detect objects is

Table 1. Some Typical Commands

Component Name	Specifications
Mac	2.6 GHz 6-Core Intel Core i7, 16GB RAM
Asus TUF A15	: AMD Ryzen 5 4600H with Radeon Graphics, 3000 Mhz, 6 Core(s)
Rapsberry Pi	Quad core Cortex-A72 (ARM v8) 64-bit SoC @ 1.8GHz

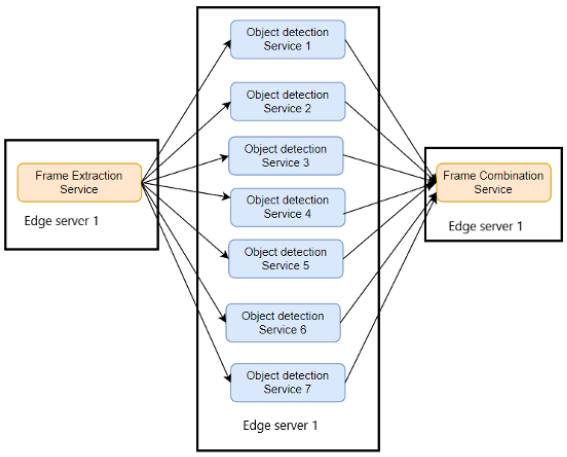


Figure 3. All On the Edge Server Architecture

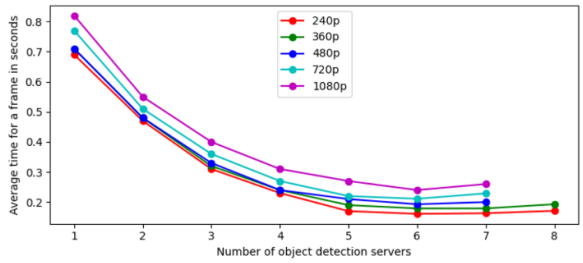


Figure 4. All on the Edge Server Results

approximately 0.71 seconds. This time reduces significantly when we increase the number of object detection containers to two, with the processing time being reduced to 0.48 seconds. This improvement in processing time is quite remarkable, and it highlights the advantages of utilizing multiple object detection containers to perform the task efficiently.

Moreover, as we continue to increase the number of object detection containers, the processing time required to detect objects decreases even further. For instance, with three object detection containers in use, the average time required to detect objects is reduced to 0.33 seconds. Similarly, when four object detection containers are used, the processing time required for object detection reduces even further to a mere 0.24 seconds.

This trend indicates that the system's processing capabilities can be effectively increased by adding more object detection containers, leading to a reduction in the time required to detect objects in frames. However, this correlation between the number of object detection containers and the processing time is not infinite, and after reaching a certain threshold, adding additional object detection servers does not significantly decrease the detection time any further.

In fact, if we continue to add more object detection servers beyond that threshold point, the processing time increases, indicating that the system has reached its optimal efficiency level. Hence, it is crucial to identify the ideal number of object detection containers required for our system to operate efficiently.

As we delve deeper into the graphical data, we can focus our attention on the specific instance of 240p. In this scenario, we can observe that the optimal threshold is achieved when we have six containers. After this point, adding one more object detection container does not cause a significant decrease in processing time, and it is evident that the system has reached its maximum capacity.

Furthermore, we can also notice that adding additional object detection containers actually results in a slight increase in the time taken to detect objects, indicating that the system is beginning to reach its limit. Therefore, it is essential to carefully monitor and optimize the number of object detection containers used in our system to ensure efficient and effective performance.

4.2.2 Case 2: Distribute Across Computing Nodes. Figure 5 depicts the way to distribute object detection service across computing nodes. Considering that distributing service across different computing nodes adds extra communication costs, a careful balance between performance improvement and increased communication delays is needed to fully realize the performance benefits of task distribution.

By increasing the number of containers on each computing node, we get Figure 6. The data illustrates a consistent trend across all video qualities—low, medium, and high—where the average processing time decreases with an increase in the number of object detection containers, ranging from one to twelve. This reduction implies that a greater number of containers enables faster parallel processing of frames since each container handles a smaller segment of the video. However, the graph also highlights a crucial observation: there exists a threshold for each video quality, beyond which adding more containers fails to further reduce the average processing time. Specifically, this threshold is met at ten

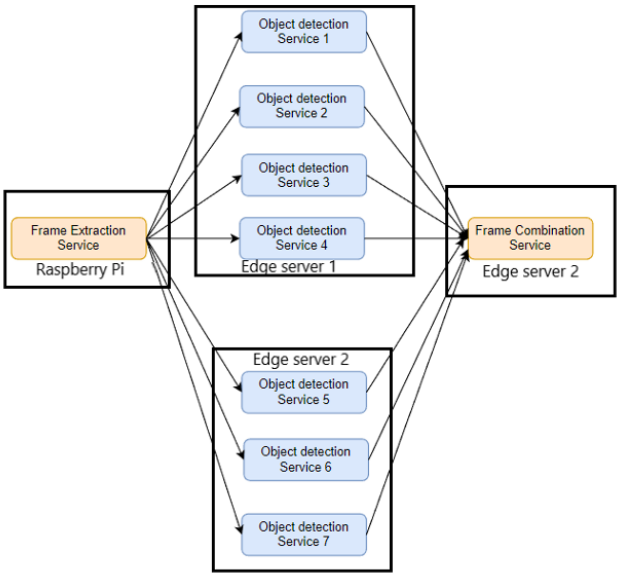


Figure 5. Distribute Across Computing Nodes Architecture

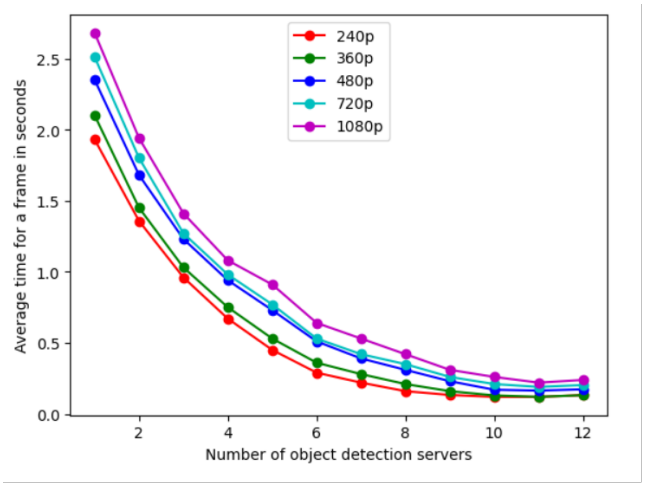


Figure 6. Distribute Across Computing Nodes Results

containers for low-quality video and eleven containers for medium and high-quality video. Beyond to this point, exceeding these container counts leads to an increase in average processing time rather than a decrease. This phenomenon suggests that there is a limit to how much parallelization can enhance processing efficiency, indicating that other factors might come into play, potentially hindering the process's overall performance.

The quality of the video itself is a significant factor affecting our results. The graph clearly illustrates that as the video quality increases from low to medium to high, the average processing time substantially rises for each number of containers. This implies that higher-quality videos require more time for processing compared to lower-quality ones, regardless of the number of containers available. There are several potential explanations for this observation. Firstly, higher-quality videos tend to have larger file sizes and resolutions, leading to increased transfer times between services via REST calls over the network. Secondly, these videos contain more intricate details and complex frames, prolonging the time required for frame extraction and combination. Additionally, higher-quality videos demand more computational and memory resources from the object detection model, leading to extended inference times and the production of accurate results.

4.3 Best Scenario

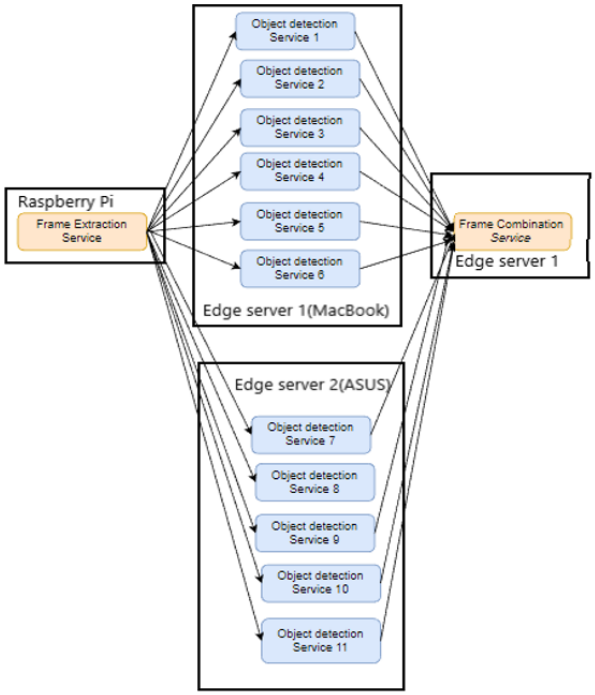


Figure 7. Best Output scenario

When we have a Frame extraction service on Raspberry Pi and six object detection and one frame combination services on MacBook Pro and five object detection services on Asus, we get the lowest average time to detect frames and the time is 0.20.

5 Conclusion

Deploying AI applications at the edge needs a careful balance between the performance improvement due to parallel computation across different computing nodes and the increased communication delays due to data movement needed to fully realize the performance benefits of task distribution. We propose a way to solve this problem by building a dynamic distributed scheduler that schedules different computing tasks

of an application across different computing nodes based on the current computing and networking conditions with the goal of meeting all the requirements of the application, including latency, privacy, power, and cost constraints. This scheduler constantly monitors the current compute and network conditions and dynamically adjusts the scheduling of the compute tasks accordingly. One important feature of this system is device filing. In order to address the heterogeneity among different computing devices, each device in the IoT environment is profiled for different computing tasks at different computing loads[12].

To apply this dynamic scheduler to a more practical scenario, we design a distributed system to support video stream object detection applications. To get performance improvement, we propose to deploy this application across different computing nodes. To get each computing node's computing capacity, we evaluate them under different scenarios. With each device profiling, the dynamic scheduler could manage AI computing tasks to meet constraints from the global level. To the best of our knowledge, this is the first scheduler that addresses these challenges for scheduling tasks at the edge.

Our next step is to Integrate this augmented reality application with our dynamic scheduler. We will also integrate FPGA and GPU in this distributed edge AI system.

Acknowledgments

This research is supported by grants from NSF.

References

[1] Ishfaq Ahmad, Sanjay Ranka, and Samee Ullah Khan. 2008. Using game theory for scheduling tasks on multi-core processors for simultaneous optimization of performance and energy. In *2008 IEEE international symposium on parallel and distributed processing*. IEEE, 1–6.

[2] Luca Bertuccelli, Han-Lim Choi, Peter Cho, and Jonathan How. 2009. Real-time multi-UAV task assignment in dynamic and uncertain environments. In *AIAA guidance, navigation, and control conference*. 5776.

[3] Matthew Bradbury, Arshad Jhumka, and Tim Watson. 2021. Trust trackers for computation offloading in edge-based IoT networks. In *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*. IEEE, 1–10.

[4] Víctor Campos, Francesc Sastre, Maurici Yagües, Míriam Bellver, Xavier Giró-i Nieto, and Jordi Torres. 2017. Distributed training strategies for a computer vision deep learning algorithm on a distributed GPU cluster. *Procedia Computer Science* 108 (2017), 315–324.

[5] Keyan Cao, Yefan Liu, Gongjie Meng, and Qimeng Sun. 2020. An overview on edge computing research. *IEEE access* 8 (2020), 85714–85728.

[6] Shiyu Chang, Wei Han, Jiliang Tang, Guo-Jun Qi, Charu C Aggarwal, and Thomas S Huang. 2015. Heterogeneous network embedding via deep architectures. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*. 119–128.

[7] Xiang Chen and Zhuwei Qin. 2020. Exploring decentralized collaboration in heterogeneous edge training. In *2020 IEEE/ACM Symposium on Edge Computing (SEC)*. IEEE, 450–453.

[8] Jeffrey Dean, Greg Corrado, Rajat Monga, Kai Chen, Matthieu Devin, Mark Mao, Marc'aurelio Ranzato, Andrew Senior, Paul Tucker, Ke Yang, et al. 2012. Large scale distributed deep networks. *Advances in neural information processing systems* 25 (2012).

[9] Arpita Ghosh, Mohammad Mahdian, Daniel M Reeves, David M Pennock, and Ryan Fugger. 2007. Mechanism design on trust networks. In *International Workshop on Web and Internet Economics*. Springer, 257–268.

[10] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. 2017. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*. 2961–2969.

[11] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. 2019. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*. 1314–1324.

[12] Fei Hu, Kunal Mehta, Shivakant Mishra, and Mohammad AlMutawa. 2023. A Dynamic Distributed Scheduler for Computing on the Edge. arXiv:2308.06806 [cs.DC]

[13] Shi Jin, Zhaofeng Guo, Dongli Liu, Yanhua Yang, et al. 2022. A study on the application of distributed system technology-guided machine learning in malware detection. *Computational Intelligence and Neuroscience* 2022 (2022).

[14] Hakima Khelifi, Senlin Luo, Boubakr Nour, Akrem Sellami, Hassine Moungla, Syed Hassan Ahmed, and Mohsen Guizani. 2018. Bringing deep learning at the edge of information-centric internet of things. *IEEE Communications Letters* 23, 1 (2018), 52–55.

[15] Wenbin Li and Matthieu Liewig. 2020. A survey of AI accelerators for edge environment. In *Trends and Innovations in Information Systems and Technologies: Volume 2 8*. Springer, 35–44.

[16] Di Liu, Hao Kong, Xiangzhong Luo, Weichen Liu, and Ravi Subramaniam. 2022. Bringing AI to edge: From deep learning's perspective. *Neurocomputing* 485 (2022), 297–320.

[17] Tamás Nepusz and Tamás Vicsek. 2012. Controlling edge dynamics in complex networks. *Nature Physics* 8, 7 (2012), 568–573.

[18] Mitra Pooyandeh and Insoo Sohn. 2021. Edge network optimization based on ai techniques: A survey. *Electronics* 10, 22 (2021), 2830.

[19] Weisong Shi, Jie Cao, Quan Zhang, Youhuizi Li, and Lanyu Xu. 2016. Edge computing: Vision and challenges. *IEEE internet of things journal* 3, 5 (2016), 637–646.

[20] Sucheta Soundarajan, Tina Eliassi-Rad, Brian Gallagher, and Ali Pinar. 2017. ε-wgx: Adaptive edge probing for enhancing incomplete networks. In *Proceedings of the 2017 ACM on Web Science Conference*. 161–170.

[21] Joost Verbraeken, Matthijs Wolting, Jonathan Katzy, Jeroen Kloppenburg, Tim Verbelen, and Jan S Rellermeyer. 2020. A survey on distributed machine learning. *Acm computing surveys (csur)* 53, 2 (2020), 1–33.

[22] Bin Yang, Manohar Kaul, and Christian S Jensen. 2013. Using incomplete information for complete weight annotation of road networks. *IEEE Transactions on Knowledge and Data Engineering* 26, 5 (2013), 1267–1279.

[23] Wei Zheng, Qi Zhu, Marco Di Natale, and Alberto Sangiovanni Vincentelli. 2007. Definition of task allocation and priority assignment in hard real-time distributed systems. In *28th IEEE International Real-Time Systems Symposium (RTSS 2007)*. IEEE, 161–170.

[24] Junlong Zhou, Tongquan Wei, Mingsong Chen, Jianming Yan, Xiaobo Sharon Hu, and Yue Ma. 2015. Thermal-aware task scheduling for energy minimization in heterogeneous real-time MPSoC systems. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 35, 8 (2015), 1269–1282.

[25] Qi Zhu, Haibo Zeng, Wei Zheng, Marco DI Natale, and Alberto Sangiovanni-Vincentelli. 2013. Optimization of task allocation and priority assignment in hard real-time distributed systems. *ACM Transactions on Embedded Computing Systems (TECS)* 11, 4 (2013), 1–30.